Retry

Retry · Publish event despite transient broker errors · Build with Waqar

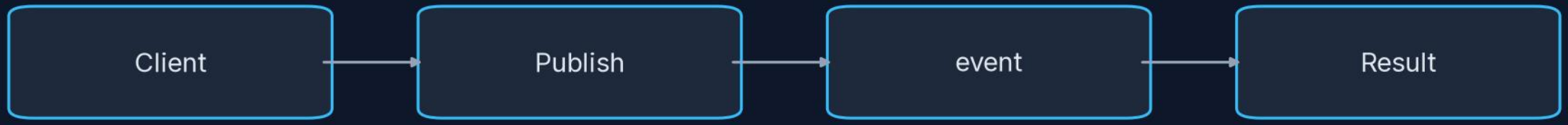
Today's Hook

- Today's force: Publish event despite transient broker errors
- We earn Retry by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Retry

The Problem

- Scenario: Publish event despite transient broker errors
- Constraints: solve «Publish event despite transient broker errors» with Retry — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Publish event despite transient broker errors
- Success looks like: caught=broker

Structure



Retry: Publish event despite transient broker errors

Real-World Scenario

- Running example: Publish event despite transient broker errors
- Key types: Retry
- Happy-path stdout cue: caught=broker
- Failure-path stdout cue: caught=broker

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.retry.problem;

import java.util.function.Supplier;

public final class NaivePublish {
  public static String once(Supplier<String> s){ return s.get(); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Publish event despite transient broker errors
- Without Retry, call sites absorb the wrong details.
- Failure mode to show on camera: caught=broker
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.retry.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class Retry {
  static String call(int maxAttempts, Supplier<String> action){
    RuntimeException last=null;
    for (int i=1;i<=maxAttempts;i++){
      try { return action.get(); } catch (RuntimeException e){ last=e; }
    }
    throw last;
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.retry.problem;

import java.util.concurrent.atomic.AtomicInteger;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Retry: transient broker errors ===");
    System.out.println("--- naive ---");
    try { NaivePublish.once(() -> { throw new RuntimeException("broker"); }); } catch(Exception e){ System.out.println("caught="+e.getMessage()); }
    System.out.println("--- retry ---");
    AtomicInteger n=new AtomicInteger();
    System.out.println(Retry.call(3, () -> { if(n.incrementAndGet()<3) throw new RuntimeException("broker"); return "published"; }));
    try { Retry.call(2, () -> { throw new RuntimeException("still"); }); } catch(RuntimeException e){ System.out.println("caught="+e.getMessage()); }
  }
}
```

Walkthrough Output

```
=== Retry: transient broker errors ===
--- naive ---
caught=broker
--- retry ---
published
caught=still
```

Step-by-Step Trace

- Match stdout to stages — look for: caught=broker
- Then the injected failure: caught=broker
- Re-run: java -cp ... 31-retry ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Retry
// Prefer records + sealed where they shrink the demo.
// Problem under test: Publish event despite transient broker errors
```

Trade-offs

- Use Retry when «Publish event despite transient broker errors» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Retry in product language.

Related Patterns

- Name the pattern usually confused with Retry and separate the forces.
- Sketch the roles for «Publish event despite transient broker errors» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Publish event despite transient broker errors
- Naive pain → Retry boundary → Demo stdout.
- Remember the failure cue: caught=broker
- Break a ProblemTest assertion, watch it fail, restore it.